Searching Algorithms
Algorithms

Abdullah All Mamun Siam

Lecturer, Dhaka International University(DIU)

Contents

Linear Search Algorithm

Linear search is a simple search algorithm used to find a particular value in a list or an array. It works by sequentially checking each element of the array until a match is found or the end of the array is reached.

The algorithm for linear search is relatively simple.

Step 1: Start at the first element (index 0) of the array.
Step 2: Compare the target value (key) with the value at the current position.
Step 3: If they are the same, return the position of the match.
Step 4: If they are not the same, move to the next element and repeat the comparison.
Step 5: Continue this process until a match is found or the entire array has been checked.
Step 6: If no match is found, print a message saying the element is not in the array and stop the program.

Time and Space Complexity of Linear Search Algorithm

Best Case: O(1) Target is the first element

Worst Case: O(n) Target is the last element or not present

Average Case: O(n) Assuming uniform distribution of the target

Space Complexity: O(1) No extra space is used

Advantages of the Linear Search Algorithm

Simplicity: Linear search is easy to understand and implement.

No need for sorting: The data does not need to be sorted for linear search to work, unlike some other search algorithms (e.g., binary search).

Handles all data types: Linear search works on both primitive (e.g., integers, strings) and non-primitive (e.g., objects, custom data structures) data types.

Small dataset: It is efficient for small dataset.

Low memory usage: It has O(1) space complexity since it doesn’t require additional memory aside from the input data.

Disadvantages of the Linear Search Algorithm

Inefficient for Large Datasets: Linear search has a time complexity of O(n), meaning it becomes slow as the size of the dataset increases.

Binary Search

Binary search is a search algorithm used to find the position of a target value within a sorted array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty.

To apply Binary Search algorithm:

The data structure must be sorted.

Binary Search Algorithm

Binary Search Algorithm Steps:

Below is the step-by-step algorithm for Binary Search:

1.Find the middle index of the search space.
2.Compare the middle element with the target key:

- If they match, return the index (found).
- If the key is smaller, search in the left half.
- If the key is larger, search in the right half.

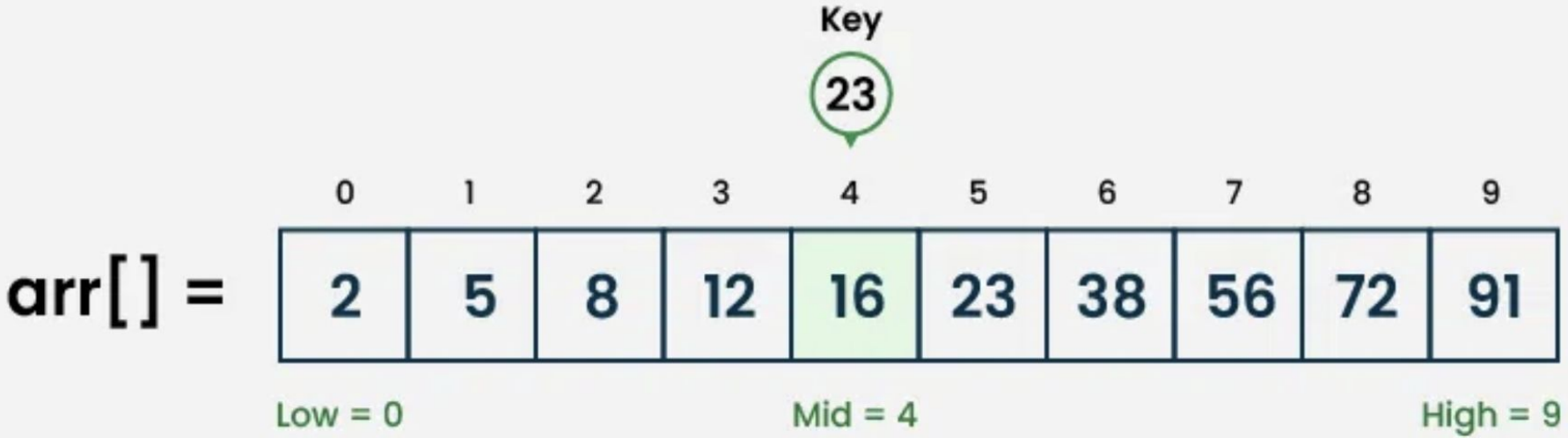
3.Repeat the process on the selected half until the key is found or no elements remain.
4.If the key is not found, return -1 (not found).

Binary Search Algorithm

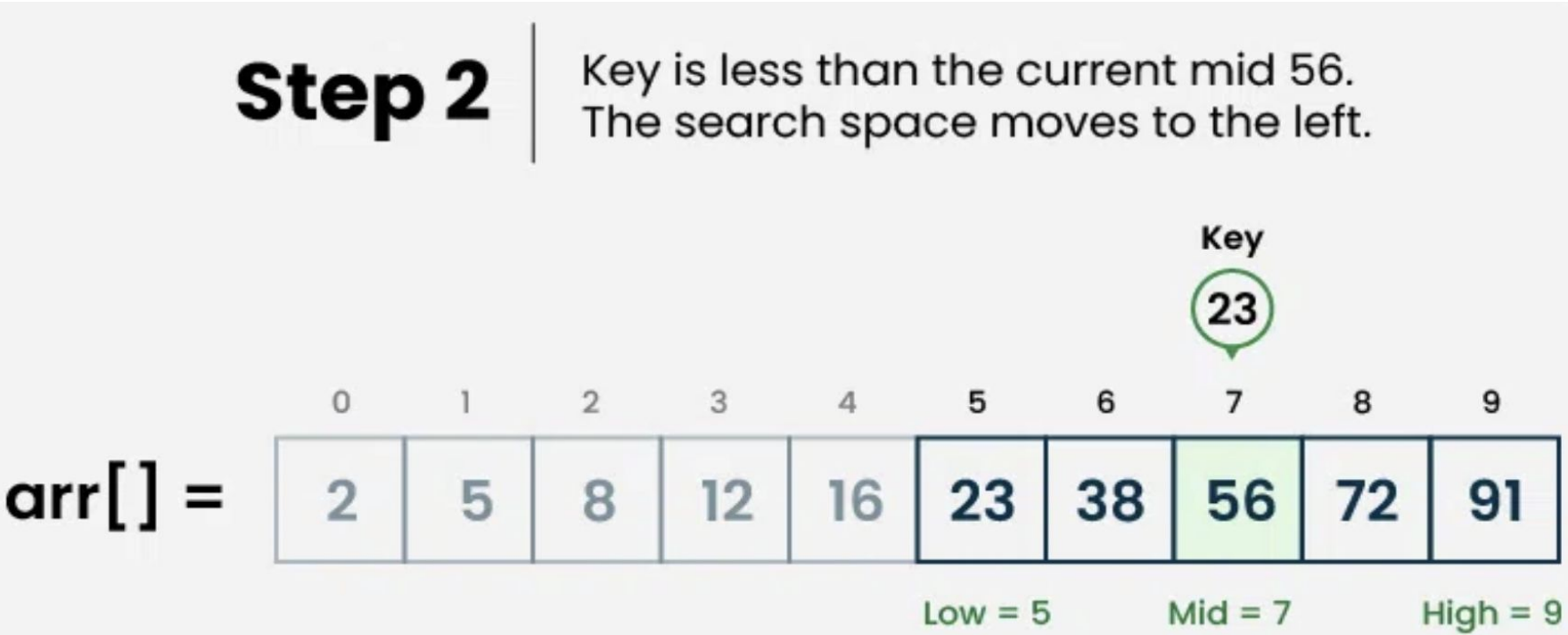
Initially | Find Key = 23 using Binary Search

Binary Search Algorithm

Step 1 | Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.



Binary Search Algorithm



Binary Search Algorithm

Step 3 | If the key matches the value of the mid element, the element is found and stop search.

Complexity Analysis of Binary Search Algorithm

Time Complexity:

Best Case: O(1)

Average Case: O(log N)

Worst Case: O(log N)

Auxiliary Space: O(1)

Advantages of Binary Search

1.Binary search is faster than linear search, especially for large arrays.

2.More efficient than other searching algorithms with a similar time complexity.

3.Binary search is well-suited for searching large datasets that are stored in external memory.

Disadvantages of Binary Search

- The array should be sorted

Ternary Search

Ternary search is a search algorithm that is used to find the position of a target value within a sorted array. It operates on the principle of dividing the array into three parts instead of two, as in binary search. The basic idea is to narrow down the search space by comparing the target value with elements at two points that divide the array into three equal parts.

mid1 = l + (r-l)/3

mid2 = r – (r-l)/3

Ternary Search Algorithm

Below are the step-by-step explanation of working of Ternary Search:

Step 1: Set two pointers, left and right, initially pointing to the first and last elements of our search space.

Step 2: Calculate two midpoints, mid1 and mid2, dividing the current search space into three roughly equal parts:

mid1 = left + (right – left) / 3

mid2 = right – (right – left) / 3

Step 3: If the target is equal to the element at mid1 or mid2, the search is successful, and the index is returned

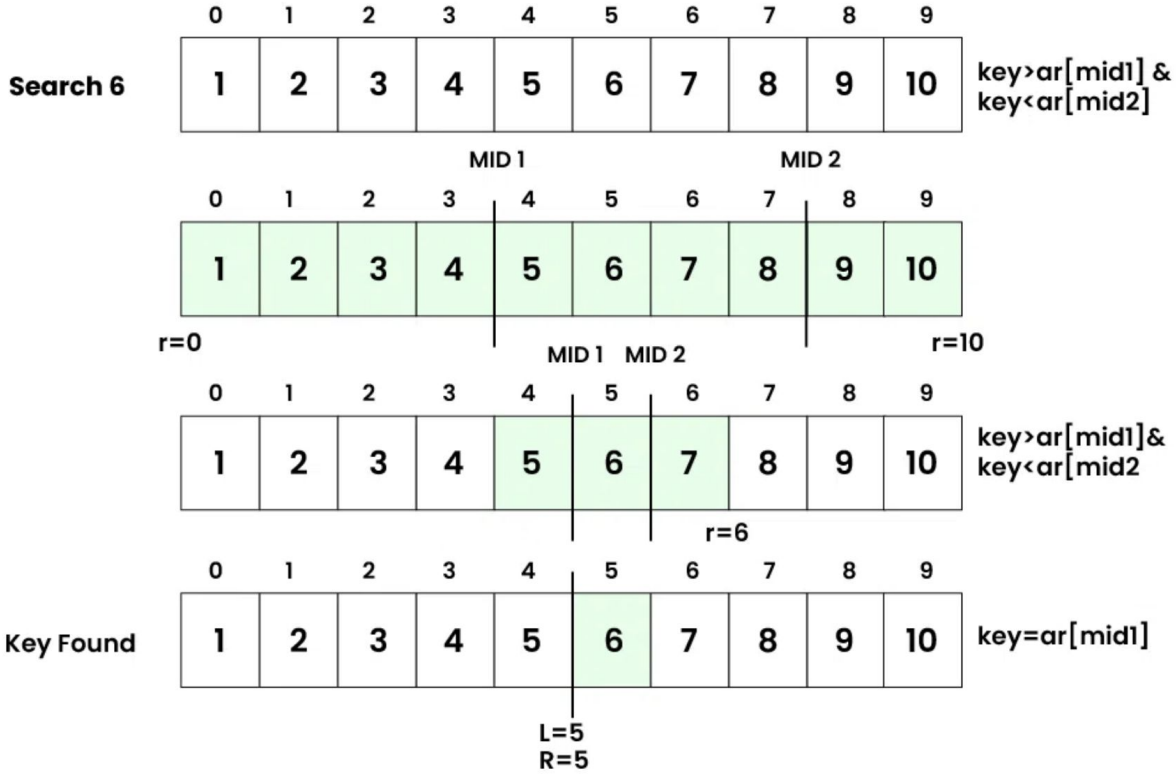
If the target is less than the element at mid1, update the right pointer to mid1 – 1.

If the target is greater than the element at mid2, update the left pointer to mid2 + 1.

If the target is between the elements at mid1 and mid2, update the left pointer to mid1 + 1 and the right pointer to mid2 – 1.

Step 4: Repeat the process with the reduced search space until the target is found or the search space becomes empty.If the search space is empty and the target is not found, return a value indicating that the target is not present

Ternary Search Algorithm



Complexity Analysis of Ternary Search

Time Complexity:

Worst case: O(log3N)

Average case: Θ(log3N)

Best case: Ω(1)

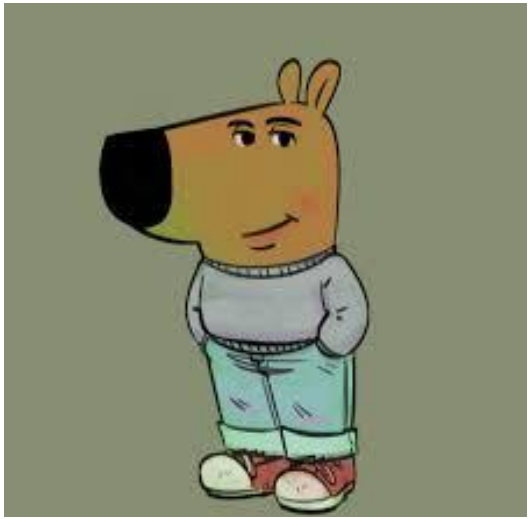
Space Complexity:

Auxiliary Space: O(1)

Binary search vs Ternary search

Binary search is generally preferred over ternary search due to its slightly lower time complexity. While both have logarithmic time complexity, binary search divides the search space into two parts, reducing the number of comparisons and operations needed. Ternary search, dividing into three parts, may involve more computations.

Hence, in practice, binary search is often more efficient and easier to implement.



Now I am a chill guy.
Thanks